

Récupérer des données sur Internet

Table des matières

Requête HTTP	2
Requête asynchrone : AJAX	2
Qu'est-ce qu'un service REST?	2
Exemple complet	3
Charger et afficher des images depuis Internet	8
Exercices	11
Références	11

Requête HTTP

Le protocole HTTP (Hypertext Transfer Protocol) est un protocole de communication client-serveur développé pour le Web. Ce protocole de la couche application utilise, dans les faits, le protocole TCP comme couche de transport.

Les clients HTTP les plus connus sont les navigateurs Web permettant à un utilisateur d'accéder à un serveur contenant les données. Un serveur HTTP utilise par défaut le port 80 (443 pour HTTPS).

Dans le protocole HTTP, il existe un mécanisme pour échanger des données entre le navigateur et le serveur Web. Ce mécanisme s'appelle requête HTTP. Il existe deux types de requêtes : synchrones et asynchrones

Dans ce chapitre, nous abordons le concept de requêtes asynchrones.

Requête asynchrone : AJAX

Dans une application Web, la méthode classique de dialogue entre un navigateur et un serveur est la suivante : lors de chaque manipulation faite par l'utilisateur, le navigateur envoie une requête contenant une référence à une page Web, puis le serveur Web effectue des calculs, et envoie le résultat sous forme d'une page Web à destination du navigateur. Celui-ci affichera alors la page qu'il vient de recevoir. Chaque manipulation entraîne la transmission et l'affichage d'une nouvelle page. L'utilisateur doit attendre l'arrivée de la réponse pour effectuer d'autres manipulations.

En utilisant AJAX, le dialogue entre le navigateur et le serveur se déroule la plupart du temps de la manière suivante : un programme écrit en langage de programmation JavaScript, incorporé dans une page web, est exécuté par le navigateur. Celui-ci envoie en arrière-plan des demandes au serveur Web, puis modifie le contenu de la page actuellement affichée par le navigateur Web en fonction du résultat reçu du serveur, évitant ainsi la transmission et l'affichage d'une nouvelle page complète.

La méthode classique de dialogue utilise des mécanismes propres au World Wide Web, qui sont incorporés dans tous les navigateurs ainsi que les robots d'indexation, et ne nécessite pas de programmation. Au contraire, le fonctionnement d'AJAX nécessite de programmer en JavaScript les échanges entre le navigateur et le serveur Web. Il nécessite également de programmer les modifications à effectuer dans la page Web à la réception des réponses, sans quoi les dialogues se font à l'insu de l'utilisateur.

En AJAX, comme le nom l'indique, les demandes sont effectuées de manière asynchrone: le navigateur Web continue d'exécuter le programme JavaScript alors que la demande est partie, il n'attend pas la réponse envoyée par le serveur Web et l'utilisateur peut continuer à effectuer des manipulations pendant ce temps.

Qu'est-ce qu'un service REST?

Les services HTTP peuvent être développées en utilisant REST (Representational State Transfer). REST est devenu une convention pour développer des services Web REST utilise le protocole HTTP pour effectuer les opérations suivantes sur les données:

- Création (Create)
- Lecture (Read)
- Mise à jour (Update)
- Suppression (Delete)

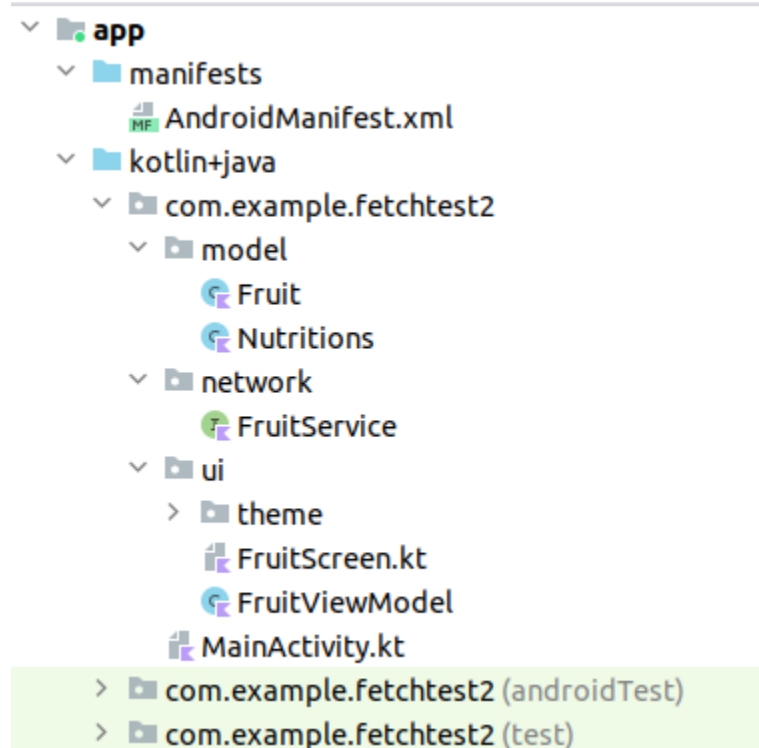
Ce sont des opérations nommées CRUD pour Create, Read, Update et Delete. Le type de requête HTTP détermine le type d'opération à exécuter. Chaque requête HTTP possède un verbe ou une méthode qui détermine son intention. Voici le standard des méthodes HTTP:

- GET (Retourner les données);
- POST (Créer les données);
- PUT (Mettre à jour les données);
- DELETE (Supprimer les données);

Exemple complet

Pour cet exemple une application affichant des informations sur les fruits sera réalisé. Les informations proviendront de l'API REST <https://www.fruityvice.com/>.

1. Créer un nouveau projet avec le modèle *Empty Activity*. Nommez ce projet **FetchTest2**. Votre projet aura l'architecture suivante:



2. Créer le modèle de données: **model/Fruit.kt** et **model/Nutritions.kt**

```

1 package com.example.fetchtest2.model
2
3 data class Fruit(
4     var name: String,
5     var id: Int,
6     var family: String,
7     var genus: String,
8     var order: String,
9     var nutritions: Nutritions
10 )

```

```

1 package com.example.fetchtest2.model
2
3 data class Nutritions(
4     var carbohydrates: Double,
5     var protein: Double,
6     var fat: Double,
7     var calories: Double,
8     var sugar: Double
9 )

```

3. Ajouter les librairie *Retrofit* et *gson* dans `build.gradle (Module:app)`.

```

1 // Ajout pour Retrofit et Gson
2 implementation "com.squareup.retrofit2:retrofit:2.9.0"
3 implementation "com.squareup.retrofit2:converter-scalars:2.9.0"
4 implementation "com.squareup.retrofit2:converter-gson:2.9.0"
5 implementation "com.google.code.gson:gson:2.8.5"

```

4. Autoriser l'application à accéder à l'Internet en modifiant le fichier `AndroidManifest.xml`

```

1 <!--permissions for INTERNET-->
2 <uses-permission android:name="android.permission.INTERNET"/>

```

5. L'interface: `network/FruitService.kt`

```

1 package com.example.fetchtest2.network
2
3 import com.example.fetchtest2.model.Fruit
4 import retrofit2.Call
5 import retrofit2.http.GET
6 import retrofit2.http.Path
7
8 interface FruitService {
9     @GET("api/fruit/{fruit}")
10     fun getData(@Path("fruit") fruit: String): Call<Fruit>
11 }

```

6. Création de la vue `ui/ConvScreen.kt`

```

1 @OptIn(ExperimentalMaterial3Api::class)
2 @Composable
3 fun FruitScreen(modifier: Modifier = Modifier){
4     var entree: String = ""
5     Column(){
6         TextField(
7             value = entree,
8             onValueChange = { entree = it },
9             label = { Text(text = "Pouces: ") }
10        )
11        Button(onClick = {
12            // Action ici
13        }){
14            Text(text = "Afficher")
15        }
16        Text(
17            text = "Famille: ",
18        )
19    }
20 }

```

7. Le fichier `MainActivity.kt` est maintenant simplement composé de:

```

1 package com.example.fetchtest2
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.fillMaxSize
7 import androidx.compose.material3.MaterialTheme
8 import androidx.compose.material3.Surface
9 import androidx.compose.material3.Text
10 import androidx.compose.runtime.Composable
11 import androidx.compose.ui.Modifier
12 import androidx.compose.ui.tooling.preview.Preview
13 import com.example.fetchtest2.ui.FruitScreen
14 import com.example.fetchtest2.ui.theme.FetchTest2Theme
15
16 class MainActivity : ComponentActivity() {
17     override fun onCreate(savedInstanceState: Bundle?) {
18         super.onCreate(savedInstanceState)
19         setContent {
20             FetchTest2Theme {
21                 Surface(
22                     modifier = Modifier.fillMaxSize(),
23                     color = MaterialTheme.colorScheme.background
24                 ) {
25                     FruitScreen()
26                 }
27             }
28         }
29     }
30 }

```

8. Vérifier le fonctionnement de votre interface utilisateur.
9. Ouvrez le fichier `build.gradle(Module: app)` dans le bloc `dependencies` et ajoutez la dépendance suivante pour `ViewModel`.

```
1 implementation "androidx.lifecycle:lifecycle-viewmodel-compose:2.5.1"
```

10. Création de la vue modèle `ui/FruitViewModel.kt` → La logique de l'application

```

1 package com.example.fetchtest2.ui
2
3 import android.util.Log
4 import androidx.compose.runtime.mutableStateOf
5 import androidx.lifecycle.ViewModel
6 import com.example.fetchtest2.model.Fruit
7 import com.example.fetchtest2.model.Nutritions
8 import com.example.fetchtest2.network.FruitService
9 import kotlinx.coroutines.flow.MutableStateFlow
10 import kotlinx.coroutines.flow.StateFlow

```

```

11 import kotlinx.coroutines.flow.asStateFlow
12 import kotlinx.coroutines.flow.update
13 import retrofit2.*
14 import retrofit2.converter.gson.GsonConverterFactory
15
16 class FruitViewModel : ViewModel(){ // Héritage
17
18     // L'état de l'interface (UI State)
19     private val _uiState = MutableStateFlow(Fruit("", 0, "???", "", "",
Nutritions(0.0,0.0,0.0,0.0,0.0)))
20     val uiState: StateFlow<Fruit> = _uiState.asStateFlow()
21
22     // information provenant de la zone de texte
23     private var _nomFruit = mutableStateOf("")
24
25     fun updateNomFruit(nom: String){
26         _nomFruit.value = nom
27     }
28
29     fun getNomFruit(): String{
30         return _nomFruit.value
31     }
32
33     // Action du bouton
34     fun afficher(){
35         val retrofit: Retrofit = Retrofit.Builder().baseUrl
("https://www.fruityvice.com/")
36         .addConverterFactory(GsonConverterFactory.create()).build()
37         val service: FruitService = retrofit.create(FruitService::class.
java)
38         val requete = service.getData(_nomFruit.value)
39         requete.enqueue(object: Callback<Fruit>{
40             override fun onResponse(
41                 call: Call<Fruit>,
42                 response: Response<Fruit>
43             ){
44                 var data = response.body() as Fruit
45                 Log.d("allo", "${data.family}")
46                 _uiState.update{
47                     currentState ->
48                     currentState.copy(
49                         family = data.family
50                     )
51                 }
52             }
53             override fun onFailure(call: Call<Fruit>, t: Throwable) {
54                 Log.e("Error", t.stackTraceToString())
55             }
56         })
57     })
58
59 }
60 }

```

Explication du fetch

11. Modifiez la vue (`ui/FruitScreen.kt`) afin de faire le lien avec le modèle de vue.

```
1 package com.example.fetchtest2.ui
2
3 import androidx.compose.foundation.layout.Column
4 import androidx.compose.material3.Button
5 import androidx.compose.material3.ExperimentalMaterial3Api
6 import androidx.compose.material3.Text
7 import androidx.compose.material3.TextField
8 import androidx.compose.runtime.Composable
9 import androidx.compose.runtime.collectAsState
10 import androidx.compose.ui.Modifier
11 import androidx.lifecycle.viewmodel.compose.viewModel
12
13 @OptIn(ExperimentalMaterial3Api::class)
14 @Composable
15 fun FruitScreen (fruitViewModel: FruitViewModel = viewModel(), modifier:
    Modifier = Modifier){
16     var entree: String = ""
17     val fruitUiState = fruitViewModel.uiState.collectAsState()
18     Column(){
19         TextField(
20             value = fruitViewModel.getNomFruit(),
21             onChange = { fruitViewModel.updateNomFruit(it) },
22             label = { Text(text = "Nom: ") }
23         )
24         Button(onClick = {
25             fruitViewModel.afficher()
26         }) {
27             Text(text = "Afficher")
28         }
29         Text(
30             text = "Famille: ${fruitUiState.value.family}",
31         )
32     }
33 }
```

Charger et afficher des images depuis Internet

Pour cet exemple une application affichant des... Pour simplifier l'intégration des images, la bibliothèque Coil ainsi que le composant AsyncImage seront utilisés.

Coil est une bibliothèque développée par la communauté, pour télécharger, mettre en mémoire tampon, décoder et mettre en cache vos images.

1. Créer un nouveau projet avec le modèle *Empty Activity*. Nommez ce projet **FetchImage`**.
2. Import the Jetpack Compose extension library:


```
implementation("io.coil-kt:coil-compose:2.4.0")
```

3. Autoriser l'application à accéder à l'Internet en modifiant le fichier `AndroidManifest.xml`

```
1 <!--permissions for INTERNET-->  
2 <uses-permission android:name="android.permission.INTERNET"/>
```

To load an image, use the `AsyncImage` composable:

```

package com.example.fetchimage

import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.Modifier
import androidx.compose.ui.tooling.preview.Preview
import coil.compose.AsyncImage
import com.example.fetchimage.ui.theme.FetchImageTheme

class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContent {
            FetchImageTheme {
                // A surface container using the 'background' color from the
theme
                Surface(
                    modifier = Modifier.fillMaxSize(),
                    color = MaterialTheme.colorScheme.background
                ) {
                    AffImage()
                }
            }
        }
    }
}

@Composable
fun AffImage(modifier: Modifier = Modifier) {
    AsyncImage(
        model = "https://openweathermap.org/img/wn/10d@2x.png",
        contentDescription = null,
    )
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    FetchImageTheme {
        AffImage()
    }
}

```

Exercices

Références

- [Récupérer des données sur Internet](#)
- [Charger et afficher des images depuis Internet](#)

1 **Mettre** le code ici